DATA 441/641 · Applied Natural Language Processing · Week 1

1.3 A Python Tour for NLP

Types, strings, containers, control flow, functions, files, NumPy, and a small class.

These notes walk through the lecture slides. The original deck is unchanged; use **(slides)** on the course hub if you want that PDF.

1. Why Python here

This course uses **Python**. Labs and homework are **Jupyter** notebooks. Submit an HTML export to Canvas (File → Download as → HTML).

Install a current Python, then Jupyter (Anaconda or `pip install notebook` is fine). You will also need `numpy` and `nltk` for Lab 1.

Python indexes from **0**. The first element of a list is `x[0]`.

---

2. Native types

Type	Example	Notes
`bool`	`False`	`True` / `False`
`int`, `float`	`42`, `12.245`	Numbers
`str`	`"Hello there"`	Unicode text
`list`	`[1, 2, 3]`	Ordered, mutable
`tuple`	`(1, 2, 3)`	Ordered, **immutable**
`set`	`{1, 2, 3}`	Unique, unordered
`dict`	`{"a": 2, "b": 3}`	Key → value

A tuple does not support `t[0] = 2`. Build a new tuple instead.

---

3. Strings

```
my_string = "Hi-Bye"
my_string[1]          # "i"
"hello" + " " + "world"
my_string[:2] + my_string[3:]
my_string.upper()
my_string.find("e")   # -1 if missing
```

Slices use `start:stop`. The stop index is **not** included. Negative indices count from the end: `my_string[-5:]`.

The `string` module is the alphabet and punctuation you will filter with:

```
import string
string.ascii_lowercase   # a–z
string.ascii_uppercase
string.digits
string.punctuation
string.whitespace
```

Useful methods: `split`, `join`, `strip`, `replace`, `lower`, `isalpha`, `isnumeric`. Format strings with `str.format`:

```
"{0}, {1}, {2}".format("a", "b", "c")
"{lat}, {lon}".format(lat="37.24N", lon="-115.81W")
```

---

4. Lists, tuples, sets, dictionaries

```
my_list = [1, 2, 3, 4]
my_list[2]
my_list.append(5)
my_list.extend([6, 7])
my_list.remove(2)     # first matching value
my_list.pop(0)        # by position
my_list.sort()
my_list.reverse()
len(my_list)
"b" in ["a", "b"]
["a", "b"] + ["c"]
```

```
my_tuple = (1, 2, 3)
my_tuple = my_tuple + (4,)
my_tuple.count(3)
my_tuple.index(4)
```

```
my_set = {1, 2, 3}
my_set.add(4)
my_set.update([5, 6])
my_set.remove(2)      # error if missing
my_set.discard(3)     # quiet if missing
{1, 2, 3}.union({3, 4})
{1, 2, 3}.intersection({3, 4})
```

```
my_dict = {"a": 1, "b": 2}
my_dict["a"]
my_dict["c"] = 3
my_dict.pop("a")
del my_dict["b"]
my_dict.keys()
my_dict.values()
```

Keys must be immutable (strings, numbers, tuples). Values can be anything.

Casting changes type: `float(1)`, `set([1, 1, 2, 3])` which is `{1, 2, 3}` because a set drops duplicates. `list + list` and `list * n` grow a list; there is no `-` or `/` for lists.

---

5. Conditionals and loops

```
my_num = 4
if my_num < 0:
    print("Negative")
elif my_num == 0:
    print("Zero")
else:
    print("Positive")
```

`if` / `elif` / `else` need a colon and an indented block.

```
a = 0
while a < 5:
    print(a)
    a += 1
```

A `while` that never updates the condition runs forever.

```
for i in range(1, 5):
    print(i)
```

`range(1, 5)` is 1, 2, 3, 4.

---

6. Functions

```
def add_two(x):
    y = x + 2
    return y

def generate_tuple(x):
    y = x * 2
    z = y * 2
    return (y, z)
```

Name arguments. Return something you can test. A function that only prints is harder to reuse.

```
def squaring_dct(n):
    if n < 1 or not isinstance(n, int):
        return "Input must be a positive integer."
    dct = {}
    for i in range(1, n + 1):
        dct[i] = i ** 2
    return dct

squaring_dct(4)
# {1: 1, 2: 4, 3: 9, 4: 16}
```

---

7. Files

```
with open("nemo.txt", "r", encoding="utf-8") as f:
    lines = f.readlines()
```

`with` closes the file even if the next line errors. `readlines()` is a list of strings, each usually ending in `\n`. `split()` on a line gives words.

Download nemo.txt and put it in the same folder as the notebook, or pass a relative path.

---

8. NumPy in one minute

Lab 1 needs a 2-D array.

```
import numpy as np

A = np.random.rand(6, 6)     # uniform on [0, 1)
A[2, :]                      # third row (index 2)
A[:, ::2]                    # every other column
A.shape                      # (6, 6)
```

`np.random.normal(0, 1, (m, n))` draws from (N(0, 1)).

---

9. A small class

A **class** is a blueprint. An **object** is one instance. Lab 1 asks for this shape:

```
class StringAnalyzer:
    def __init__(self, content):
        self.content = content

    def analyze_string(self):
        vowels = "aeiouAEIOU"
        counts = {"vowels": 0, "consonants": 0, "others": 0}
        for char in self.content:
            if char.isalpha():
                if char in vowels:
                    counts["vowels"] += 1
                else:
                    counts["consonants"] += 1
            else:
                counts["others"] += 1
        return counts

analyzer = StringAnalyzer("Hello, World! 123")
analyzer.analyze_string()
```

`__init__` runs when you call the class. Methods take `self` as the first argument. Store data on `self`.

---

10. Practice

1. Build `{"a": 1, "b": 2, "c": 3}`, drop `"a"`, and print the remaining keys.
2. Write `add_two` and check `add_two(2) == 4`.
3. Write `squaring_dct(4)` without looking. What should happen if `n` is `3.2`?
4. Open `nemo.txt` and print the first line. Do not hard-code a `C:/Users/...` path.